Flow-Pattern Anomaly Detection at the IoT Gateway

Madhav Dogra

1 Aim

Detect bad traffic from **smart-home IoT devices** (cameras, sensors, smart bulbs, voice assistants, thermostats, doorbells, smart plugs) at the home gateway. Catch known attacks and zero-days. Works on encrypted traffic. Runs under 10 ms on Raspberry-Pi-class hardware.

2 Why the gateway

Smart-home IoT endpoints can't run an agent — weak firmware, no patches, no compute. Gateway is the choke point. Every flow passes through. Only place where modern compute can sit.

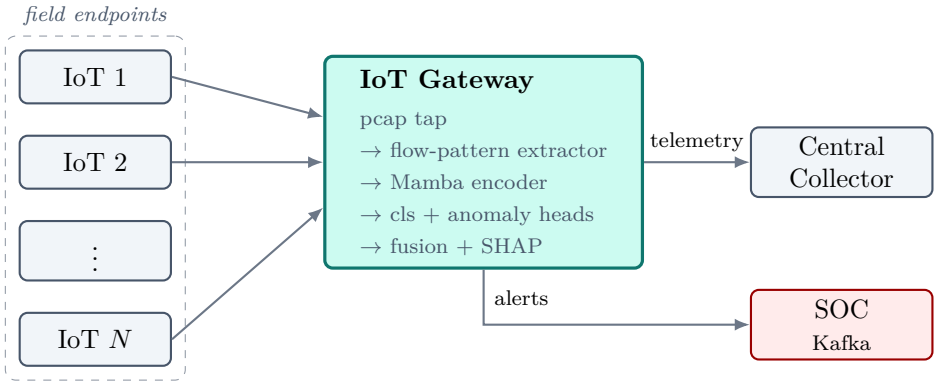

Figure 1: *

Detector lives inside the gateway. Telemetry passes through untouched; alerts go out-of-band to the SOC.

3 The input: flow patterns, not aggregates

Standard IDS reduces each flow to ∼80 summary stats. Order is gone. Attack signatures live in order. Instead — keep the first $K \approx 32$ packets of the flow as a sequence. Each packet p_t = size, direction, IAT, flags, header bytes. Payload-free, so encrypted traffic still works.

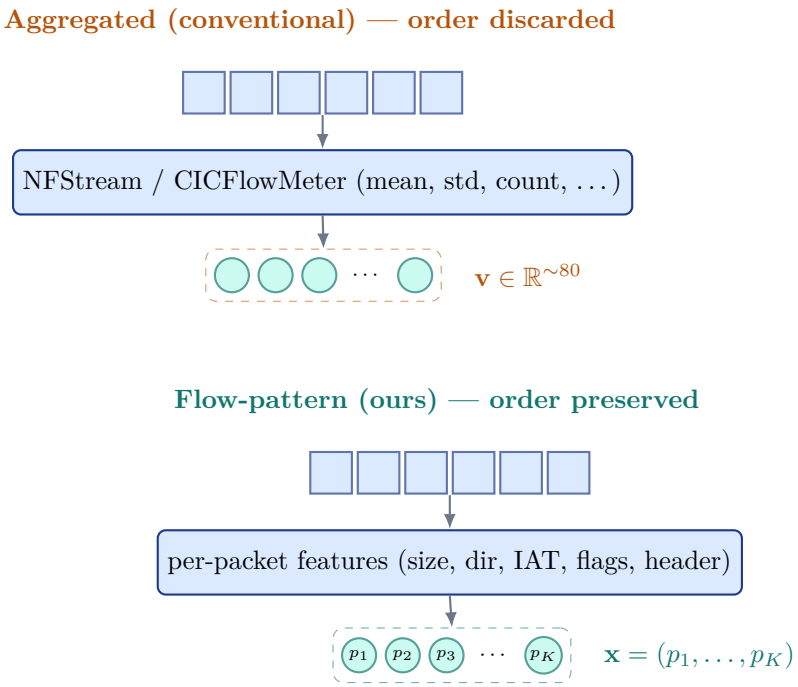

Figure 2: *

Top: collapse to summary stats, lose temporal structure. Bottom: keep the packets in order. Sequence model does the aggregation.

4 The model

- One **Mamba encoder**. Pre-trained self-supervised on benign-only flows. Produces an embedding **z**.
- Two heads share it. **Classifier** for 7 known classes reads **z** directly. **Anomaly head** (deep SVDD) reads a learned **projection** $\mathbf{z}_{\text{ano}}$ in a lower-dimensional space — one-class methods are sensitive to the curse of dimensionality, so we project down first.
- Either head fires ⇒ alert. SHAP attributions over the sequence. Sent to SOC over Kafka.
- Why Mamba and not a transformer? ET-BERT works but is ∼100M params — doesn't fit a gateway. NetMamba shows you get ET-BERT-class accuracy on a Mamba backbone at a fraction of the cost. Linear-time inference in K.

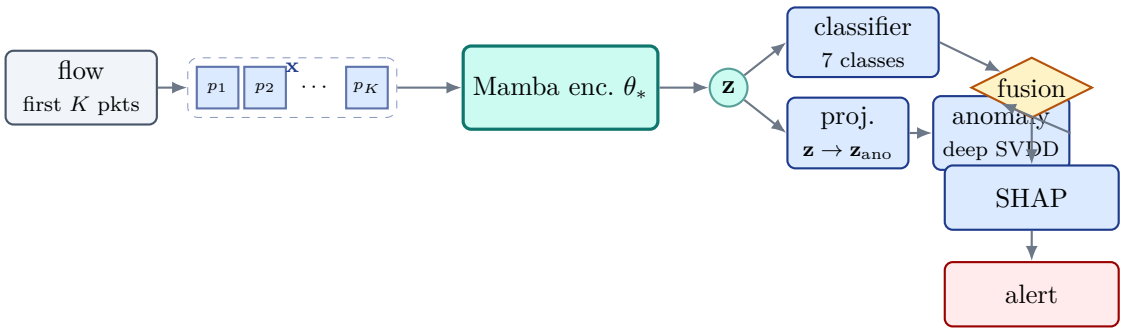

Figure 3: *

One encoder, two heads. Anomaly head sees a lower-dim projection of **z**. *Either fires ⇒ alert. SHAP explains it.*

5 Training: three stages

1. **Pre-train.** Mamba on unlabelled benign flows. Masked-pattern objective. No attack labels.

2. **Fine-tune.** Init from pre-trained weights. Joint train encoder + classifier on labelled CICIoT2023. Focal loss for imbalance.
3. **Anomaly head.** Freeze encoder. Embed a held-out benign set. Fit one-class scorer on those embeddings. Threshold at 99th percentile (1% FPR target).

Zero-day test: leave-one-class-out — hide attack classes from the supervised head and check the anomaly head catches them.

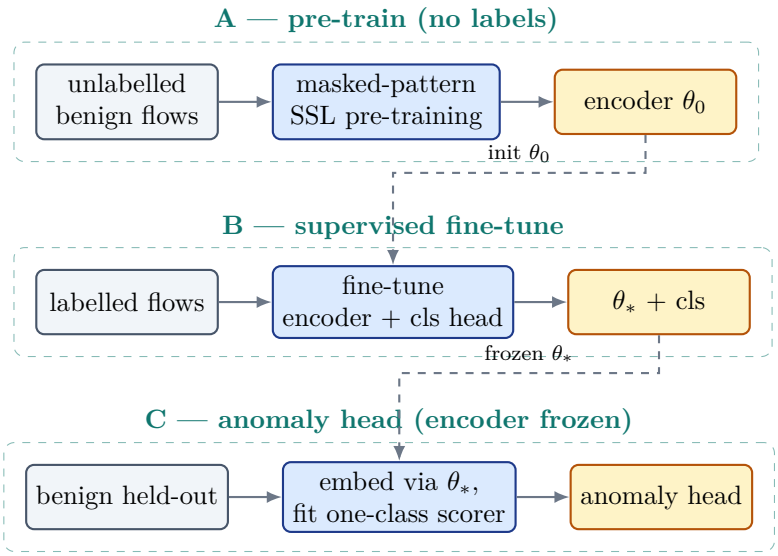

Figure 4: *

Three stages. Encoder built without labels, specialised with labels, then anomaly head added on top.

6 Two modes — default + strong (admin-switched)

Gateway runs in one of two configurations. Admin picks. Not a per-flow decision.

- **Default** (always on, low power). Small K, smallest Mamba, anomaly head only, no SHAP. Target <2 ms / flow. Continuous operation on Raspberry-Pi-class hardware.
- **Strong** (admin-switched, full pipeline). $K = 32$, full encoder, both heads (with projection on the anomaly side), SHAP on every alert. Target <10 ms / flow. Switched on for elevated threat, incident response, or high-value windows.

Same model artifacts in both modes — what differs is which components are engaged and at what size.

7 Progressive learning — new attacks

When a new attack type appears:

1. **Day-zero**: anomaly head flags it (it just looks unlike normal).
2. **Label**: SOC confirms and labels a sample.
3. **Update head only**: add a class to the classifier softmax, fine-tune just the head on the new data with a replay buffer. **Encoder stays frozen.** Takes minutes, not days.
4. **Recalibrate threshold** on a fresh benign set if needed.

Full encoder retraining is only needed if *benign* traffic itself shifts. "What's normal" (encoder, expensive) and "what's malicious" (heads, cheap) are decoupled — that's the whole point of the shared-encoder design.

8 Evaluation

- **Datasets.** CICIoT2023 (primary). TON_IoT (cross-dataset). IoT-23 (malware).

- **Baselines.** ET-BERT (upper bound, gateway-infeasible). CNN-BiLSTM on aggregated stats (representation control). **XGBoost** (classical-floor challenge — "do we need deep learning at all?"; the deep model earns its complexity only on zero-day, cross-dataset, and adversarial metrics, where XGBoost can't compete). Isolation Forest (classical unsupervised floor).
- **Metrics.** Per-class P/R/F1. PR-AUC. p50/p95/p99 latency. Model size after int8 quantisation.
- **Adversarial.** FGSM on the sequence. Plot detection rate vs ε. Both heads.
- **Edge benchmark.** Raspberry-Pi-class hardware. Target <10 ms p95 per flow.

9 Deliverables

Code repo. Pre-trained encoder checkpoint. Benchmark table (5 baselines $\times$ 3 datasets). FGSM curves. Edge-hardware latency profile. Written report.

10 Risks — and what I do about them

- Pre-training compute blows budget $\rightarrow$ warm-start from a public Mamba checkpoint.
- Class imbalance hides minority-class failure $\rightarrow$ focal loss + per-class metrics.
- Cross-dataset accuracy drop $\rightarrow$ measure honestly, don't hide.
- Edge memory overrun $\rightarrow$ int8 quantise; fall back to smaller config; reduce K.
- Byte-level adversarial sensitivity $\rightarrow$ FGSM study quantifies it; SHAP fingerprint as secondary defence.

11 Stretch goals

- Swap Mamba $\rightarrow$ Mamba-2 (SSD) if latency headroom allows.
- Add E-GraphSAGE branch over a per-window device graph.
- Federated pre-training across multiple sites.

12 Key references

ET-BERT (arxiv 2202.06335) • NetMamba (2405.11449) • Mamba (2312.00752) • Mamba-2 / SSD (2405.21060) • S4D (2206.11893) • CICIoT2023 (MDPI Sensors) • ET-SSL (Nature Sci.Rep.) • E-GraphSAGE (2103.16329).